Reviewer Pack — Monotone CLIQUE Lower Bound via GF(2) Rank Defect

Entry 550befe6a1bf · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-29 03:21:03 UTC

Monotone CLIQUE Lower Bound via GF(2) Rank Defect

Entry ID: `550befe6a1bf` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-04-29 03:21:03 UTC

1. Conjecture

Field A (mathematical branch): matroid theory **Field B** (complexity object): polymatroid expansion

Statement:

> For any monotone DNF formula F representing the k-CLIQUE_n function, the GF(2) rank defect Δ(F) = log₂(m+1) − log₂(rank_{F₂}(A_F)) satisfies Δ(F) ≥ Ω(n), while for any DNF of size poly(n), Δ(F) = O(log n)

Rationale (proposer's reasoning):

> The GF(2) rank defect captures the combinatorial complexity of DNF representations through matroid-theoretic lens. For k-CLIQUE, the exponential number of terms forces a large rank defect, while polynomial-size DNFs have low defect due to efficient span coverage. This measure aligns with GCT's orbit closure methods by quantifying structural rigidity.

Taxonomy category: `GCT` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `45e69807bc44f622`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from itertools import combinations

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    for i in range(m):
        max_row = i
        for j in range(i+1, m):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        for j in range(i+1, m):
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]

def rank(A):
    A_copy = [row[:] for row in A]
    gaussian_elimination(A_copy)
    rank = 0
    for row in A_copy:
        if any(row):
            rank += 1
    return rank

def generate_k_clique_dnf(n, k):
    clauses = []
    for clique in combinations(range(n), k):
        clause = [f"x{i+1}" for i in clique]
        clauses.append(" or ".join(clause))
    dnf = " and ".join(clauses)
    return dnf

def generate_random_dnf(n, m):
    variables = [f"x{i+1}" for i in range(n)]
    clauses = []
    for _ in range(m):
        clause = random.sample(variables, random.randint(1, n))
        if random.choice([True, False]):
            clause = [f"not {v}" for v in clause]
```

```
        clauses.append(" or ".join(clause))
    dnf = " and ".join(clauses)
    return dnf

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        if n == 20:  # Skip n=20 to avoid excessive computation time
            continue
        for _ in range(5):  # Sample 5 instances per n
            dnf = generate_k_clique_dnf(n, n // 2) if n >= 10 else generate_random_dnf(n, random.r
            A_F = []
            for clause in dnf.split(" and "):
                row = [1] * (n + 1)
                for var in clause.split(" or "):
                    if var.startswith("not "):
                        j = int(var[4:]) - 1
                        row[j] = 0
                    else:
                        j = int(var[1:]) - 1
                        row[j] = 1
                A_F.append(row)
            rank_F2 = rank(A_F)
            m = len(A_F)
            delta_F = math.log2(m + 1) - math.log2(rank_F2)
            total_metric_value += delta_F
            instances_tested += 1

    mean_metric_value = total_metric_value / instances_tested
    support_fraction = instances_tested / (len(n_values) * 5)

    if support_fraction < 0.8:
        conjecture_holds = False
        counterexample = "support_fraction_below_80"

    return {
        "metric_name": "GF(2) rank defect",
        "metric_value": mean_metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else list(range(2, 53))
    results = []

    for seed in seeds:
        result = run_trial(seed)
```

```
        print(f"TRIAL: {{\"seed\": {seed}, **{result}**}}")
        results.append(result)

    total_metric_value = sum(r["metric_value"] for r in results)
    instances_tested = sum(r["instances_tested"] for r in results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={total_metric_value / instances_tested} std=0.0 support_fra
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"support_fraction_below_80\" first_failing_seed=
    else:
        print("RESULT: INCONCLUSIVE support_fraction_below_80")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_21558f1d.py", line 109, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_21558f1d.py", line 76, in run_trial
    j = int(var[4:]) - 1
        ^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'x2'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed due to invalid literal conversion, preventing data collection to assess support fraction or counterexamples. | next: Fix the test code to handle variable parsing errors and re-run trials with multiple seeds

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	48048
2	preregistration	ollama_remote	qwen3:8b	0	0	13009

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
3	novelty	ollama_remote	qwen3:8b	0	0	25998
4	novelty	ollama_remote	qwen3:8b	0	0	20410
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12318
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12249
7	judge	ollama_remote	qwen3:8b	0	0	16557

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 148590 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/550befe6a1bf.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/550befe6a1bf.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/550befe6a1bf.tar.gz` (if generated)